REGULATORY & SUPPORT COPILOT · PART GUIDE 4 OF 5

Part 4: Support Agent and Orchestrator (Workflow B)

You build the Support Triage agent that diagnoses bank tickets, and the orchestrator that runs every task, step by step, for both workflows.

Owner	(write your name)
Your folder	`part4_support/` (inside it: `triage.py`, `prompts/`, `orchestrator/`, `tests/`) · shared with Part 5: `tests/integration/`
Your reviewer	Part 5 owner · you review Part 3
Read first	00-Project-Guide.pdf (Sections 3, 4, 8, 11, 12, 14)

Contents

1. Your role
2. What you deliver
3. Dependencies
4. Step-by-step plan
5. The Support Triage agent
6. The orchestrator
7. Quality: how you will be measured
8. How your work shows on the website
9. Testing and definition of done
10. GitHub issues to create
11. Starter prompt for your AI tool
12. Common mistakes

1. Your role

Two jobs that make you the team's **integrator**:

1. **Support Triage agent (Workflow B):** read a bank's ticket, find the transaction in the logs, identify the rejection reason, gather evidence, and draft a clear reply for a human to approve.
2. **Orchestrator:** the "manager" that receives a task from the website, runs the right agents in order in the background, records each step's status and output, writes audit events, and stores everything in SQLite so the website can show progress live.

> **Guide's note:** You are where the parts meet, so you will see integration problems first. Don't fix other people's code yourself; open an Issue for them with the exact error and input. Your first-integration demo in week 4 is the most important milestone of the project.

2. What you deliver

#	Deliverable	Location
D1	Support Triage agent	`part4_support/triage.py`, `part4_support/prompts/triage.md`
D2	Task store (SQLite)	`part4_support/orchestrator/store.py`
D3	Orchestrator: start tasks, run steps in background, update status	`part4_support/orchestrator/orchestrator.py` (re-exported in `part4_support/orchestrator/__init__.py`)
D4	Command-line runner	`part4_support/orchestrator/run.py`
D5	Integration tests for both workflows (with Part 5)	`tests/integration/`
D6	Tests, READMEs, AI_LOG	`part4_support/tests/`, `part4_support/README.md`, `part4_support/AI_LOG.md`
D7	Your contract models: `TriageResult`, `TaskStep`, `Task`	`contracts/schemas.py` (Contract Day)

3. Dependencies

You use	From	Until ready, use
`llm.complete_json()`, `search()`	Part 2	`fake` provider and search stub
`search_logs()`, `get_ticket()`, `list_tickets()`	Part 1	stubs + first data sample
`summarize()`, `analyze()`, `draft()`	Part 3	their stubs
`audit.log_event()`, `mask()`	Part 5	stubs (do nothing / return text)

Uses your work	How
Part 5 (website)	`start_regulation_task()`, `start_ticket_task()`, `get_task()`, `list_tasks()`, `set_task_status()`, `update_output()`
Part 5 (evaluation)	`run_regulation_sync()` and `run_ticket_sync()`, the same steps without a background thread

4. Step-by-step plan

Week 1

1. Environment setup. Read the whole Project Guide carefully; you need the full picture more than anyone.
2. Write `TriageResult`, `TaskStep` and `Task` for Contract Day. Propose the `outputs` keys (Section 6.3) to the team.

Week 2

3. Merge stubs: `triage()` returns the example; the orchestrator functions exist and return a fake finished task.
4. Implement `part4_support/orchestrator/store.py` for real (it has no dependencies).

Week 3

5. Implement the real orchestrator with background threads, using the other parts' stubs. Test with `python -m part4_support.orchestrator.run`.
6. Implement the triage agent with the `fake` provider.

Week 4: first integration (milestone M2)

7. Connect the real agents of Parts 1–3 as they become ready. Sit with Part 5 and make both workflows run from the website with the `fake` provider.
8. Write the two integration tests.

Weeks 5–6

9. Switch to a real model. Run all 20 tickets, compare with `ticket_eval.json`, and improve the triage agent.
10. Error handling: a failing step marks the task failed with a readable message; a **Retry** runs the task again.

Weeks 7–8

11. Report section, demo rehearsal, swap session.

5. The Support Triage agent

```
triage(ticket: Ticket) -> TriageResult
```

Steps (code first, LLM where needed):

1. **Find the transaction reference.** First with code: regular expressions for patterns such as `payment 7731`, `ref 7731`, `ref=7731`, `transfer #7731`. Only if code finds nothing, ask the LLM to extract it (or say there is none). Use `ticket.transaction_ref` if it is already set.
2. **Classify the ticket.** If there is no reference and the text is a general question (for example about cut-off times), take the **question path**: answer from `search(ticket.body, filters={"doc_type": "product_doc"})`, with `reason_code = null`.
3. **Search the logs (code).** `search_logs(ref=ref, client_id=ticket.client_id)`. Keep the matching entries as `evidence`.
   - No entries found → `escalate = true`, `confidence = "low"`, and a reply that politely says the team is checking the reference.
   - The last status is settled → explain that the payment settled successfully (with the time).
   - An `RJCT` entry → take its `reason_code`.
4. **Gather knowledge (RAG).** `search(reason_code, filters={"doc_type": "reason_codes"}, k=2)` for the meaning and the recommended action, plus `search(ticket.body, filters={"doc_type": "past_ticket"}, k=2)` for similar past cases. Put them in `sources`.
5. **Ask the LLM** for `root_cause` (one or two sentences) and `draft_reply`, giving it the ticket, the log evidence and the sources inside tags. Reply rules (write them in `prompts/triage.md`): polite and short; explain the reason in plain words; tell the bank what to do next; never include internal system names, full account numbers or other clients' data; sign as "Support Team".
6. **Confidence (code, not LLM):** `high` = log found and reason code explained in the knowledge base · `medium` = log found but the code is unknown or there are no sources · `low` = no log found. Set `escalate = true` for `low`.

6. The orchestrator

6.1 Public functions

```
start_regulation_task(doc: RegulationDoc) -> str     # returns task_id immediately
start_ticket_task(ticket: Ticket) -> str
get_task(task_id) -> Task
list_tasks(type=None, status=None) -> list[Task]
set_task_status(task_id, status) -> None             # called by the website after review
update_output(task_id, key, value) -> None           # e.g. store an edited change request
retry_task(task_id) -> str                           # starts a new task with the same input
run_regulation_sync(doc) -> Task                     # same steps, no thread (for evaluation)
run_ticket_sync(ticket) -> Task
```

6.2 How a task runs

1. `start_…` creates a `Task` with `status = "running"`, all its steps `waiting`, saves it, writes an audit event `task_created`, starts a background thread (`threading.Thread(daemon=True)`), and returns the `task_id` (for example `T-20260302-0007`).
2. The thread runs the steps in order. For each step: set it `running` and save → call the agent → save the output under its key → set it `done` with timing → write an audit event `step_done` (actor = agent name, details = short summary such as "found 3 changes").

3. If a step raises an exception: set it `failed` with the error message, set the task `failed`, write an audit event `step_failed`, and stop.
4. After the last step: set the task `pending_review`. From now on, only the website (human decisions) changes the status.

6.3 Steps and output keys

Task type	Steps (names shown on the website)	`outputs` keys
regulation	`Regulation Watcher` → `Impact Analysis` → `Change Request`	`summary`, `impact_report`, `change_requests`
ticket	`Support Triage`	`triage`

Store outputs as `model_dump(mode="json")` so they are plain JSON. Tip: to show finer progress for tickets, the triage agent can accept an optional `on_progress(message)` callback that the orchestrator saves as the step's latest message ("Searching logs …").

6.4 The task store (`part4_support/orchestrator/store.py`)

- SQLite file from `settings.app_db`. One table `tasks(task_id TEXT PRIMARY KEY, type TEXT, status TEXT, created_at TEXT, data TEXT)`, where `data` holds the whole `Task` as JSON.
- Functions: `save_task(task)`, `load_task(task_id)`, `query_tasks(type=None, status=None)` (newest first).
- **Open a new connection in every function** (SQLite connections must not be shared between threads), and use `with conn:` so each write is committed.
- Part 5 keeps its audit and review tables in the same database file but in their own tables; agree on table names to avoid clashes.

6.5 Command-line runner

```
python -m part4_support.orchestrator.run --regulation REG-003
python -m part4_support.orchestrator.run --ticket TCK-0007
python -m part4_support.orchestrator.run --list
```

It runs the task synchronously and prints each step and the final outputs. Everyone on the team uses it to test the whole chain without the website.

7. Quality: how you will be measured

Part 5's evaluation compares triage results with Part 1's `ticket_eval.json`:

Metric	Target
Reference found correctly	≥ 95%

Metric	Target
Reason code correct	≥ 90% (it comes from logs, so code should make it near perfect)
Escalation correct (missing references escalate, others don't)	100%
Root cause contains the expected keywords	≥ 80%
Reply contains no full account number (masking check)	100%
Average time per ticket	Report it (no target)

8. How your work shows on the website

- The **Task page** step timeline (waiting / running / done / failed, with durations) comes straight from your `TaskStep`s; the website polls `get_task()` every 2 seconds.
- The **triage card** shows reason code, root cause, evidence log lines, sources, confidence badge, an escalation warning, and the draft reply with Approve / Edit / Reject.
- The **Dashboard** lists tasks from `list_tasks()`.
- The **Retry** button calls `retry_task()`.

9. Testing and definition of done

- [ ] Triage unit tests (fake provider, mocked logs and search): rejected payment, settled payment, missing reference, general question.
- [ ] Reference-extraction tests for at least 6 writing styles.
- [ ] Confidence and escalation rules tested in code.
- [ ] Store tests: save/load round trip, filters, newest first.
- [ ] Orchestrator tests: steps go waiting → running → done; a failing agent marks the task failed with its message; background start returns immediately.
- [ ] Integration tests (with Part 5): both workflows end in `pending_review` with all outputs, using the fake provider.
- [ ] `python -m part4_support.orchestrator.run` works for every circular and ticket with a real model.

10. GitHub issues to create

Label `part-4`:

1. Contract models: TriageResult, TaskStep, Task (+ outputs keys proposal)
2. Stubs for triage and orchestrator
3. SQLite task store
4. Orchestrator with background threads and step tracking
5. Command-line runner

6. Triage: reference extraction (regex + LLM fallback)
7. Triage: log search, evidence, question path
8. Triage: RAG sources, LLM reply, confidence rules
9. Error handling and retry
10. Integration tests for both workflows (with Part 5)
11. Quality run on 20 tickets

11. Starter prompt for your AI tool

```
You are helping me build Part 4 (Support Agent and Orchestrator) of a 5-person student
project, the "Regulatory & Support Copilot". I attached the project guide, my part guide
and contracts/schemas.py. Rules: Python 3.11, Pydantic v2, sqlite3 and threading from the
standard library; call the LLM only through part2_knowledge.common.llm; use other parts only
through their
public functions listed in Section 8.2 of the project guide; edit only part4_support/
and tests/integration/; pytest tests for every function.
First, give me a short plan. Wait for my approval. Then do this task:
<for example: implement part4_support/orchestrator/store.py as described in Section 6.4>.
```

12. Common mistakes

- **Running agents inside the web request.** The page freezes for a minute. Always use the background thread.
- **Sharing one SQLite connection between threads.** It crashes randomly. One connection per call.
- **Swallowing errors.** A failed step must show its message on the website, or nobody can debug it.
- **Letting the LLM find the reason code.** The logs already contain it; code reads it exactly.
- **Waiting for everyone before integrating.** Integrate with stubs from week 3, then swap in real parts one by one.